



API Liga Deportiva

Sports League API

Fase 1: Fundamentos y CRUD de Teams

Stack Tecnológico

.NET 8 | C# | Entity Framework Core | SQL Server
AutoMapper | Swagger | Arquitectura N-Capas

Guía paso a paso para construcción en clase

1. Objetivos de la Fase 1

Al finalizar esta fase, los estudiantes serán capaces de:

- Crear una solución .NET con múltiples proyectos organizados en capas
- Configurar Entity Framework Core con Code First y SQL Server
- Implementar una clase base abstracta (AuditBase) para campos comunes de auditoría
- Implementar el patrón Repository (genérico y específico)
- Crear DTOs con AutoMapper para separar entidades de la capa de presentación
- Implementar un CRUD completo de Teams con arquitectura N-Capas
- Configurar Swagger para documentación y pruebas de la API

2. Arquitectura del Proyecto

La solución sigue una arquitectura de N-Capas con tres proyectos independientes:

Proyecto	Capa	Responsabilidad
SportsLeague.API	Presentación	Controllers, DTOs, Mappings, Middlewares, Program.cs
SportsLeague.Domain	Negocio	Entities, Enums, Interfaces, Services (lógica de negocio)
SportsLeague.DataAccess	Datos	DbContext, Repositories (implementaciones), Migrations

Flujo de una petición HTTP

```
Request HTTP
↓
Controller (API)    → Recibe DTO, mapea a Entity, llama al Service
↓
Service (Domain)    → Ejecuta lógica de negocio, llama al Repository
↓
Repository (DataAccess) → Ejecuta queries con EF Core contra la BD
↓
SQL Server
```

Referencias entre proyectos

```
SportsLeague.API    → referencia a    SportsLeague.Domain
```

```
SportsLeague.API      → referencia a → SportsLeague.DataAccess  
SportsLeague.DataAccess → referencia a → SportsLeague.Domain
```

IMPORTANTE: SportsLeague.Domain NO referencia a ningún otro proyecto.

Principio de Dependencia

El Domain no conoce ni la API ni el DataAccess. Define interfaces (contratos) que las otras capas implementan. Esto permite cambiar la base de datos o el framework web sin tocar la lógica de negocio.

3. Prerrequisitos

Herramienta	Detalle
.NET 8 SDK	Descargar de https://dotnet.microsoft.com/download
Visual Studio 2022 / VS Code	Con la carga de trabajo ASP.NET y desarrollo web
SQL Server	Express o Developer Edition (LocalDB también funciona)
SQL Server Management Studio	Para visualizar la base de datos (opcional)
Postman / Swagger	Para probar los endpoints de la API

4. Paso a Paso: Crear la Solución

4.1 Crear la solución y los proyectos

Abrir una terminal en la carpeta donde se desea crear el proyecto y ejecutar los siguientes comandos:

Crear la carpeta y la solución

```
mkdir SportsLeague  
cd SportsLeague  
dotnet new sln -n SportsLeague
```

Crear los 3 proyectos

```
# Proyecto API (Web API)
dotnet new webapi -n SportsLeague.API -controllers

# Proyecto Domain (Class Library)
dotnet new classlib -n SportsLeague.Domain

# Proyecto DataAccess (Class Library)
dotnet new classlib -n SportsLeague.DataAccess
```

Agregar los proyectos a la solución

```
dotnet sln add SportsLeague.API/SportsLeague.API.csproj
dotnet sln add SportsLeague.Domain/SportsLeague.Domain.csproj
dotnet sln add SportsLeague.DataAccess/SportsLeague.DataAccess.csproj
```

Configurar las referencias entre proyectos

```
# API referencia a Domain
dotnet add SportsLeague.API/SportsLeague.API.csproj reference
SportsLeague.Domain/SportsLeague.Domain.csproj

# API referencia a DataAccess (para registrar servicios en Program.cs)
dotnet add SportsLeague.API/SportsLeague.API.csproj reference
SportsLeague.DataAccess/SportsLeague.DataAccess.csproj

# DataAccess referencia a Domain
dotnet add SportsLeague.DataAccess/SportsLeague.DataAccess.csproj reference
SportsLeague.Domain/SportsLeague.Domain.csproj
```

Tip para clase

Explicar a los estudiantes por qué API necesita referenciar a DataAccess: es únicamente para registrar las implementaciones concretas de los repositorios en el contenedor de inyección de dependencias en Program.cs.

Eliminar archivos por defecto

Eliminar los archivos Class1.cs que se crean automáticamente en Domain y DataAccess:

```
# Windows
del SportsLeague.Domain\Class1.cs
del SportsLeague.DataAccess\Class1.cs

# Linux / Mac
rm SportsLeague.Domain/Class1.cs
rm SportsLeague.DataAccess/Class1.cs
```

4.2 Crear la estructura de carpetas

Crear las siguientes carpetas dentro de cada proyecto:

SportsLeague.Domain

```
SportsLeague.Domain/  
├── Entities/  
├── Enums/  
├── Interfaces/  
│   ├── Repositories/  
│   └── Services/  
└── Services/
```

SportsLeague.DataAccess

```
SportsLeague.DataAccess/  
├── Context/  
├── Repositories/  
└── Migrations/      (se crea automáticamente con EF Core)
```

SportsLeague.API

```
SportsLeague.API/  
├── Controllers/  
├── DTOs/  
│   ├── Request/  
│   └── Response/  
├── Mappings/  
└── Middlewares/
```

5. Instalar Paquetes NuGet

SportsLeague.DataAccess

```
cd SportsLeague.DataAccess  
dotnet add package Microsoft.EntityFrameworkCore -v 8.0.*  
dotnet add package Microsoft.EntityFrameworkCore.SqlServer -v 8.0.*  
dotnet add package Microsoft.EntityFrameworkCore.Tools -v 8.0.*  
cd ..
```

SportsLeague.API

```
cd SportsLeague.API
dotnet add package Microsoft.EntityFrameworkCore.Design -v 8.0.*
dotnet add package AutoMapper.Extensions.Microsoft.DependencyInjection
dotnet add package Swashbuckle.AspNetCore
cd ..
```

SportsLeague.Domain

El proyecto Domain necesita el paquete de Logging para usar ILogger en los Services:

```
cd SportsLeague.Domain
dotnet add package Microsoft.Extensions.Logging.Abstractions
cd ..
```

Sobre los paquetes

EntityFrameworkCore.Design se instala en la API porque es el proyecto de inicio (startup project) desde donde se ejecutan las migraciones. AutoMapper.Extensions.Microsoft.DependencyInjection facilita el registro de AutoMapper en el contenedor de inyección de dependencias. Logging.Abstractions solo trae las interfaces (ILogger) sin implementaciones concretas, manteniendo Domain independiente.

6. Crear la Clase Base AuditBase

Antes de crear cualquier entidad, definimos una clase base abstracta que contiene los campos comunes a todas las tablas del sistema: Id, CreatedAt y UpdatedAt. Esto evita repetir estos campos en cada entidad.

SportsLeague.Domain/Entities/AuditBase.cs

```
namespace SportsLeague.Domain.Entities;

public abstract class AuditBase
{
    public int Id { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime? UpdatedAt { get; set; }
}
```

 ¿Por qué una clase abstracta?

La palabra clave `abstract` impide que se pueda instanciar `AuditBase` directamente (no queremos un `new AuditBase()`). Solo sirve como base para que otras clases hereden de ella. El campo `CreatedAt` es `required` y se asigna automáticamente al crear el registro. El campo `UpdatedAt` es `nullable` (`DateTime?`) porque al momento de la creación aún no ha habido ninguna actualización, así que su valor inicial es `null`.

Enseñanza clave: Herencia en OOP

Este es un ejemplo práctico del principio DRY (Don't Repeat Yourself). En lugar de escribir `Id`, `CreatedAt` y `UpdatedAt` en las 9 entidades del sistema, lo definimos una vez en `AuditBase` y todas las entidades lo heredan. Si mañana necesitamos agregar un campo `IsActive` a todas las tablas, solo lo agregamos en `AuditBase`.

7. Crear la Entity Team

Ahora creamos nuestra primera entidad que hereda de `AuditBase`. Notar que ya no necesitamos declarar `Id`, `CreatedAt` ni `UpdatedAt`:

SportsLeague.Domain/Entities/Team.cs

```
namespace SportsLeague.Domain.Entities;

public class Team : AuditBase
{
    public string Name { get; set; } = string.Empty;
    public string City { get; set; } = string.Empty;
    public string Stadium { get; set; } = string.Empty;
    public string? LogoUrl { get; set; }
    public DateTime FoundedDate { get; set; }
}
```

Tip

`Team` hereda de `AuditBase`, lo que significa que automáticamente tiene las propiedades `Id`, `CreatedAt` y `UpdatedAt`. El campo `LogoUrl` es `nullable` (`string?`) porque es opcional. Todas las demás entidades del proyecto (`Player`, `Match`, `Tournament`, etc.) seguirán este mismo patrón de herencia.

8. Configurar el DbContext

Crear el archivo `LeagueDbContext.cs` dentro de `SportsLeague.DataAccess/Context/`:

SportsLeague.DataAccess.Context.LeagueDbContext.cs

```

using Microsoft.EntityFrameworkCore;
using SportsLeague.Domain.Entities;

namespace SportsLeague.DataAccess.Context;

public class LeagueDbContext : DbContext
{
    public LeagueDbContext(DbContextOptions<LeagueDbContext> options)
        : base(options)
    {
    }

    public DbSet<Team> Teams => Set<Team>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<Team>(entity =>
        {
            entity.HasKey(t => t.Id);
            entity.Property(t => t.Name)
                .IsRequired()
                .HasMaxLength(100);
            entity.Property(t => t.City)
                .IsRequired()
                .HasMaxLength(100);
            entity.Property(t => t.Stadium)
                .HasMaxLength(150);
            entity.Property(t => t.LogoUrl)
                .HasMaxLength(500);
            entity.Property(t => t.CreatedAt)
                .IsRequired();
            entity.Property(t => t.UpdatedAt)
                .IsRequired(false);
            entity.HasIndex(t => t.Name)
                .IsUnique();
        });
    }
}

```

** Fluent API vs Data Annotations**

Usamos Fluent API en `OnModelCreating` para mantener las entidades limpias (sin atributos como `[Required]`). Esto separa la configuración de la base de datos de la definición del modelo de dominio. El índice único en `Name` garantiza que no haya equipos duplicados a nivel de base de datos. Notar que `UpdatedAt` se configura como `IsRequired(false)` para reflejar que es nullable.

9. Definir las Interfaces

9.1 IGenericRepository

El repositorio genérico usa la restricción where T : AuditBase para garantizar que solo se pueda usar con entidades que hereden de nuestra clase base:

SportsLeague.Domain/Interfaces/Repositories/IGenericRepository.cs

```
using SportsLeague.Domain.Entities;

namespace SportsLeague.Domain.Interfaces.Repositories;

public interface IGenericRepository<T> where T : AuditBase
{
    Task<IEnumerable<T>> GetAllAsync();
    Task<T?> GetByIdAsync(int id);
    Task<T> CreateAsync(T entity);
    Task UpdateAsync(T entity);
    Task DeleteAsync(int id);
    Task<bool> ExistsAsync(int id);
}
```

Restricción genérica

La cláusula where T : AuditBase asegura que este repositorio solo funcione con clases que hereden de AuditBase. Esto nos da acceso a las propiedades Id, CreatedAt y UpdatedAt dentro de la implementación, y previene que alguien intente usar el repositorio con una clase que no sea una entidad del dominio.

9.2 ITeamRepository

SportsLeague.Domain/Interfaces/Repositories/ITeamRepository.cs

```
using SportsLeague.Domain.Entities;

namespace SportsLeague.Domain.Interfaces.Repositories;

public interface ITeamRepository : IGenericRepository<Team>
{
    Task<Team?> GetByNameAsync(string name);
    Task<IEnumerable<Team>> GetByCityAsync(string city);
}
```

9.3 ITeamService

SportsLeague.Domain/Interfaces/Services/ITeamService.cs

```
using SportsLeague.Domain.Entities;

namespace SportsLeague.Domain.Interfaces.Services;

public interface ITeamService
{
    Task<IEnumerable<Team>> GetAllAsync();
    Task<Team?> GetByIdAsync(int id);
    Task<Team> CreateAsync(Team team);
    Task UpdateAsync(int id, Team team);
    Task DeleteAsync(int id);
}
```

10. Implementar los Repositories

10.1 GenericRepository

SportsLeague.DataAccess/Repositories/GenericRepository.cs

```
using Microsoft.EntityFrameworkCore;
using SportsLeague.DataAccess.Context;
using SportsLeague.Domain.Entities;
using SportsLeague.Domain.Interfaces.Repositories;

namespace SportsLeague.DataAccess.Repositories;

public class GenericRepository<T> : IGenericRepository<T> where T : AuditBase
{
    protected readonly LeagueDbContext _context;
    protected readonly DbSet<T> _dbSet;

    public GenericRepository(LeagueDbContext context)
    {
        _context = context;
        _dbSet = context.Set<T>();
    }

    public async Task<IEnumerable<T>> GetAllAsync()
    {
        return await _dbSet.ToListAsync();
    }
}
```

```

public async Task<T?> GetByIdAsync(int id)
{
    return await _dbSet.FindAsync(id);
}

public async Task<T> CreateAsync(T entity)
{
    entity.CreatedAt = DateTime.UtcNow;
    entity.UpdatedAt = null;
    await _dbSet.AddAsync(entity);
    await _context.SaveChangesAsync();
    return entity;
}

public async Task UpdateAsync(T entity)
{
    entity.UpdatedAt = DateTime.UtcNow;
    _dbSet.Update(entity);
    await _context.SaveChangesAsync();
}

public async Task DeleteAsync(int id)
{
    var entity = await GetByIdAsync(id);
    if (entity != null)
    {
        _dbSet.Remove(entity);
        await _context.SaveChangesAsync();
    }
}

public async Task<bool> ExistsAsync(int id)
{
    return await _dbSet.AnyAsync(e => e.Id == id);
}
}

```

💡 Enseñanza clave

Gracias a la restricción `where T : AuditBase`, podemos acceder directamente a `entity.CreatedAt`, `entity.UpdatedAt` y `e.Id` dentro del `GenericRepository`. Además, el `CreateAsync` asigna automáticamente `CreatedAt` y establece `UpdatedAt` en `null`. El `UpdateAsync` asigna `UpdatedAt` con la fecha actual. El método `ExistsAsync` usa `AnyAsync` en lugar de `FindAsync` para mayor eficiencia (solo hace `SELECT 1` en lugar de traer toda la entidad).

10.2 TeamRepository

SportsLeague.DataAccess/Repositories/TeamRepository.cs

```

using Microsoft.EntityFrameworkCore;
using SportsLeague.DataAccess.Context;
using SportsLeague.Domain.Entities;
using SportsLeague.Domain.Interfaces.Repositories;

namespace SportsLeague.DataAccess.Repositories;

public class TeamRepository : GenericRepository<Team>, ITeamRepository
{
    public TeamRepository(LeagueDbContext context) : base(context)
    {
    }

    public async Task<Team?> GetByNameAsync(string name)
    {
        return await _dbSet
            .FirstOrDefaultAsync(t => t.Name.ToLower() == name.ToLower());
    }

    public async Task<IEnumerable<Team>> GetByCityAsync(string city)
    {
        return await _dbSet
            .Where(t => t.City.ToLower() == city.ToLower())
            .ToListAsync();
    }
}

```

11. Implementar el Service

SportsLeague.Domain/Services/TeamService.cs

```

using Microsoft.Extensions.Logging;
using SportsLeague.Domain.Entities;
using SportsLeague.Domain.Interfaces.Repositories;
using SportsLeague.Domain.Interfaces.Services;

namespace SportsLeague.Domain.Services;

public class TeamService : ITeamService
{
    private readonly ITeamRepository _teamRepository;
    private readonly ILogger<TeamService> _logger;

    public TeamService(ITeamRepository teamRepository, ILogger<TeamService> logger)
    {
        _teamRepository = teamRepository;
        _logger = logger;
    }

    public async Task<IEnumerable<Team>> GetAllAsync()
    {
        _logger.LogInformation("Retrieving all teams");
    }
}

```

```
        return await _teamRepository.GetAllAsync();
    }

    public async Task<Team?> GetByIdAsync(int id)
    {
        _logger.LogInformation("Retrieving team with ID: {TeamId}", id);
        var team = await _teamRepository.GetByIdAsync(id);

        if (team == null)
            _logger.LogWarning("Team with ID {TeamId} not found", id);

        return team;
    }

    public async Task<Team> CreateAsync(Team team)
    {
        // Validación de negocio: nombre único
        var existingTeam = await _teamRepository.GetByNameAsync(team.Name);
        if (existingTeam != null)
        {
            _logger.LogWarning("Team with name '{TeamName}' already exists",
team.Name);
            throw new InvalidOperationException(
                $"Ya existe un equipo con el nombre '{team.Name}'");
        }

        _logger.LogInformation("Creating team: {TeamName}", team.Name);
        return await _teamRepository.CreateAsync(team);
    }

    public async Task UpdateAsync(int id, Team team)
    {
        var existingTeam = await _teamRepository.GetByIdAsync(id);
        if (existingTeam == null)
        {
            _logger.LogWarning("Team with ID {TeamId} not found for update", id);
            throw new KeyNotFoundException(
                $"No se encontró el equipo con ID {id}");
        }

        // Validar nombre único (si cambió)
        if (existingTeam.Name != team.Name)
        {
            var teamWithSameName = await _teamRepository.GetByNameAsync(team.Name);
            if (teamWithSameName != null)
            {
                throw new InvalidOperationException(
                    $"Ya existe un equipo con el nombre '{team.Name}'");
            }
        }

        existingTeam.Name = team.Name;
        existingTeam.City = team.City;
        existingTeam.Stadium = team.Stadium;
        existingTeam.LogoUrl = team.LogoUrl;
    }
}
```

```

        existingTeam.FoundedDate = team.FoundedDate;

        _logger.LogInformation("Updating team with ID: {TeamId}", id);
        await _teamRepository.UpdateAsync(existingTeam);
    }

    public async Task DeleteAsync(int id)
    {
        var exists = await _teamRepository.ExistsAsync(id);
        if (!exists)
        {
            _logger.LogWarning("Team with ID {TeamId} not found for deletion", id);
            throw new KeyNotFoundException(
                $"No se encontró el equipo con ID {id}");
        }

        _logger.LogInformation("Deleting team with ID: {TeamId}", id);
        await _teamRepository.DeleteAsync(id);
    }
}

```

Lógica de negocio en el Service

Notar cómo el Service valida las reglas de negocio (nombre único) y el Repository solo se encarga de la persistencia. Ya no asignamos CreatedAt ni UpdatedAt aquí porque el GenericRepository lo hace automáticamente gracias a la herencia de AuditBase.

12. Crear los DTOs

Los DTOs (Data Transfer Objects) separan lo que exponemos al frontend de nuestras entidades internas.

SportsLeague.API/DTOs/Request/TeamRequestDTO.cs

```

namespace SportsLeague.API.DTOs.Request;

public class TeamRequestDTO
{
    public string Name { get; set; } = string.Empty;
    public string City { get; set; } = string.Empty;
    public string Stadium { get; set; } = string.Empty;
    public string? LogoUrl { get; set; }
    public DateTime FoundedDate { get; set; }
}

```

SportsLeague.API/DTOs/Response/TeamResponseDTO.cs

```
namespace SportsLeague.API.DTOs.Response;

public class TeamResponseDTO
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string City { get; set; } = string.Empty;
    public string Stadium { get; set; } = string.Empty;
    public string? LogoUrl { get; set; }
    public DateTime FoundedDate { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime? UpdatedAt { get; set; }
}
```

💡 Diferencia entre Request y Response DTO

El RequestDTO no incluye Id, CreatedAt ni UpdatedAt porque esos valores los genera el sistema. El ResponseDTO incluye todo lo que el frontend necesita ver, incluyendo UpdatedAt como nullable (DateTime?). Esto evita que el frontend envíe datos que no debería controlar.

13. Configurar AutoMapper

SportsLeague.API/Mappings/MappingProfile.cs

```
using AutoMapper;
using SportsLeague.API.DTOs.Request;
using SportsLeague.API.DTOs.Response;
using SportsLeague.Domain.Entities;

namespace SportsLeague.API.Mappings;

public class MappingProfile : Profile
{
    public MappingProfile()
    {
        // Team mappings
        CreateMap<TeamRequestDTO, Team>();
        CreateMap<Team, TeamResponseDTO>();
    }
}
```

14. Crear el TeamController

SportsLeague.API/Controllers/TeamController.cs

```
using AutoMapper;
using Microsoft.AspNetCore.Mvc;
```

```

using SportsLeague.API.DTOs.Request;
using SportsLeague.API.DTOs.Response;
using SportsLeague.Domain.Entities;
using SportsLeague.Domain.Interfaces.Services;

namespace SportsLeague.API.Controllers;

[ApiController]
[Route("api/[controller]")]
public class TeamController : ControllerBase
{
    private readonly ITeamService _teamService;
    private readonly IMapper _mapper;
    private readonly ILogger<TeamController> _logger;

    public TeamController(
        ITeamService teamService,
        IMapper mapper,
        ILogger<TeamController> logger)
    {
        _teamService = teamService;
        _mapper = mapper;
        _logger = logger;
    }

    [HttpGet]
    public async Task<ActionResult<IEnumerable<TeamResponseDTO>>> GetAll()
    {
        var teams = await _teamService.GetAllAsync();
        var teamsDto = _mapper.Map<IEnumerable<TeamResponseDTO>>(teams);
        return Ok(teamsDto);
    }

    [HttpGet("{id}")]
    public async Task<ActionResult<TeamResponseDTO>> GetById(int id)
    {
        var team = await _teamService.GetByIdAsync(id);

        if (team == null)
            return NotFound(new { message = $"Equipo con ID {id} no encontrado" });

        var teamDto = _mapper.Map<TeamResponseDTO>(team);
        return Ok(teamDto);
    }

    [HttpPost]
    public async Task<ActionResult<TeamResponseDTO>> Create(TeamRequestDTO dto)
    {
        try
        {
            var team = _mapper.Map<Team>(dto);
            var createdTeam = await _teamService.CreateAsync(team);
            var responseDto = _mapper.Map<TeamResponseDTO>(createdTeam);

            return CreatedAtAction(

```

```

        nameof(GetById),
        new { id = responseDto.Id },
        responseDto);
    }
    catch (InvalidOperationException ex)
    {
        return Conflict(new { message = ex.Message });
    }
}

[HttpPut("{id}")]
public async Task<ActionResult> Update(int id, TeamRequestDTO dto)
{
    try
    {
        var team = _mapper.Map<Team>(dto);
        await _teamService.UpdateAsync(id, team);
        return NoContent();
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return Conflict(new { message = ex.Message });
    }
}

[HttpDelete("{id}")]
public async Task<ActionResult> Delete(int id)
{
    try
    {
        await _teamService.DeleteAsync(id);
        return NoContent();
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
}
}

```

Códigos HTTP correctos

GET retorna 200 (Ok), POST retorna 201 (CreatedAtAction con la URL del recurso creado), PUT/DELETE retornan 204 (NoContent), errores de validación retornan 409 (Conflict), y recursos no encontrados retornan 404 (NotFound).

15. Configurar Program.cs

Este es el punto de entrada de la aplicación donde registramos todos los servicios en el contenedor de inyección de dependencias:

SportsLeague.API/Program.cs

```
using Microsoft.EntityFrameworkCore;
using SportsLeague.DataAccess.Context;
using SportsLeague.DataAccess.Repositories;
using SportsLeague.Domain.Interfaces.Repositories;
using SportsLeague.Domain.Interfaces.Services;
using SportsLeague.Domain.Services;

var builder = WebApplication.CreateBuilder(args);

// — Entity Framework Core —
builder.Services.AddDbContext<LeagueDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

// — Repositories —
builder.Services.AddScoped(typeof(IGenericRepository<>),
    typeof(GenericRepository<>));
builder.Services.AddScoped<ITeamRepository, TeamRepository>();

// — Services —
builder.Services.AddScoped<ITeamService, TeamService>();

// — AutoMapper —
builder.Services.AddAutoMapper(typeof(Program).Assembly);

// — Controllers —
builder.Services.AddControllers();

// — Swagger —
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// — Middleware Pipeline —
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

💡 Inyección de Dependencias

AddScoped significa que se crea una nueva instancia por cada petición HTTP. Notar cómo registramos Interface → Implementación. Esto permite que el Controller reciba ITeamService sin conocer la clase concreta TeamService. El GenericRepository se registra con typeof porque es una clase genérica abierta.

16. Configurar la Cadena de Conexión

SportsLeague.API/appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=localhost;Database=SportsLeagueDb;Trusted_Connection=true;TrustServerCertificate=true;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

i Cadena de conexión

Trusted_Connection=true usa autenticación de Windows. Si los estudiantes usan SQL Server con usuario y contraseña, cambiar a: Server=localhost;Database=SportsLeagueDb;UserId=sa;Password=TuPassword;TrustServerCertificate=true;

17. Crear y Aplicar la Migración

Ejecutar los siguientes comandos desde la raíz de la solución:

Crear la primera migración

```
dotnet ef migrations add InitialCreate \
  --project SportsLeague.DataAccess \
  --startup-project SportsLeague.API
```

Aplicar la migración a la base de datos

```
dotnet ef database update \  
  --project SportsLeague.DataAccess \  
  --startup-project SportsLeague.API
```

💡 Tip para estudiantes

Si usan Visual Studio, pueden usar la Package Manager Console: Add-Migration InitialCreate -Project SportsLeague.DataAccess y Update-Database -Project SportsLeague.DataAccess. Asegúrense de que el proyecto de inicio (startup project) sea SportsLeague.API.

18. Probar la API

Ejecutar la aplicación:

```
cd SportsLeague.API  
dotnet run
```

Abrir el navegador en la URL que muestra la consola (normalmente <https://localhost:5001/swagger>) para acceder a la interfaz de Swagger.

Probar con el siguiente JSON de ejemplo en el POST /api/Team

```
{  
  "name": "Atlético Nacional",  
  "city": "Medellín",  
  "stadium": "Atanasio Girardot",  
  "logoUrl": "https://example.com/nacional.png",  
  "foundedDate": "1947-03-07"  
}
```

Verificaciones esperadas

Acción	Código HTTP	Resultado
POST /api/Team	201 Created	Retorna equipo con Id, CreatedAt tiene valor, UpdatedAt es null
GET /api/Team	200 OK	Retorna lista con el equipo creado
GET /api/Team/1	200 OK	Retorna el equipo específico
GET /api/Team/999	404 Not Found	Mensaje de error

Acción	Código HTTP	Resultado
POST duplicado	409 Conflict	Mensaje: ya existe el nombre
PUT /api/Team/1	204 No Content	Actualización exitosa, UpdatedAt ya tiene valor
DELETE /api/Team/1	204 No Content	Eliminación exitosa

19. Estructura Final del Proyecto

```

SportsLeague/
├── SportsLeague.sln
├── SportsLeague.API/
│   ├── Controllers/
│   │   └── TeamController.cs
│   ├── DTOs/
│   │   ├── Request/
│   │   │   └── TeamRequestDTO.cs
│   │   └── Response/
│   │       └── TeamResponseDTO.cs
│   ├── Mappings/
│   │   └── MappingProfile.cs
│   ├── Middlewares/
│   ├── Program.cs
│   └── appsettings.json
├── SportsLeague.Domain/
│   ├── Entities/
│   │   ├── AuditBase.cs      ← NUEVA
│   │   └── Team.cs
│   ├── Enums/
│   ├── Interfaces/
│   │   ├── Repositories/
│   │   │   ├── IGenericRepository.cs
│   │   │   └── ITeamRepository.cs
│   │   └── Services/
│   │       └── ITeamService.cs
│   ├── Services/
│   │   └── TeamService.cs
├── SportsLeague.DataAccess/
│   ├── Context/
│   │   └── LeagueDbContext.cs
│   ├── Repositories/
│   │   ├── GenericRepository.cs
│   │   └── TeamRepository.cs
│   └── Migrations/

```

20. Próxima Fase

En la Fase 2 agregaremos:

- Entity Player con relación a Team (Foreign Key) heredando de AuditBase

- IPlayerRepository y IPlayerService con sus implementaciones
- PlayerController con DTOs de Request y Response
- Navigation Properties y configuración de relaciones en EF Core
- Nueva migración para la tabla Players